

corridor

Build Your Own Bridge

Native USDC between Sui and Base on Circle CCTP V2. We build Corridor: a fee our Move contract enforces, and gas we pay for our users on both chains.

day2-corridor.vercel.app

[KEYS, DOLLARS AND BITCOIN · DAY 2 · SUIHUB LAGOS]

RURU (@MINTING_RURU)

A bridge is a burn, a proof and a mint.

CCTP does all three.
You add the fee and the gas.

CCTP burns native USDC on one chain, Circle's attesters sign the message, and the other chain mints the same amount. No wrapped token, no pool, nothing for you to guard. Your product is everything around it: the fee, the relayer, the status screen. V1 contracts are paused on 1 December 2026, so this is V2.

[CHECKED ON 8 NOVEMBER]

CCTP V2 on Sui: what is live

From Circle's docs, then confirmed on chain: objects on Sui testnet, bytecode on Base Sepolia, live burns and mints.

	What we found	How we know
Sui domain	8 (Base Sepolia 6, Ethereum Sepolia 0)	Circle docs, Iris messages
V2 on Sui testnet	Live: TMM 0x267d…3c64, MT 0xe967…bbe	GraphQL types, live burns
Sui as source	Standard only, attested in 5 s	Our timed burn C1qH…bB5T
Into Sui	Fast works, 1 bp fee	Live mint DHwH…yad
V1	Burn limits cut 31 Oct, paused 1 Dec 2026	Circle blog, 27 Aug
Fee endpoint	/v2/burn/USDC/fees rejects domain 8	Iris, 8 Nov

[THE BURN ON SUI]

Out of Sui: one PTB, four calls

V2 split the burn: CCTP checks, the USDC handler burns, CCTP sends. Into Sui is four calls too: receive_message, prepare_mint, mint, complete_mint.

01

Take the fee

corridor::send (ours). Measures the coin, puts the fee in the vault, returns Circle's burn ticket.

02

Check

Circle's deposit_for_burn, package-auth variant. Domain, limits, fees. Returns BurnReceipt and the coin.

03

Burn

handler::burn. The USDC handler burns the coin with its MintCap and returns a ticket.

04

Send

complete_burn. Sends the CCTP message and emits DepositForBurn.

05

Attest

Circle's Iris signs the message. Anyone can now mint on Base.

[MOVE/CORRIDOR/SOURCES/CORRIDOR.MOVE]

The fee, enforced by Move

Abridged. The fee is measured from the coin, never from a number the client sends.

MOVE

```
public fun send<T: drop>(
    self: &mut Corridor<T>, mut coin: Coin<T>,
    destination_domain: u32, mint_recipient: address,
    circle_max_fee: u64, min_finality_threshold: u32, ctx: &mut TxContext,
): DepositForBurnWithPackageAuthTicket<T, CorridorAuth> {
    assert!(self.is_open(), EPaused);
    let gross = coin.value();
    let fee = self.fee_for(gross);             // max(bps, floor)
    assert!(gross > fee, EAmountNotAboveFee);
    self.vault.join(coin.split(fee, ctx).into_balance());
    emit(TransferSentEvent<T> { gross, fee, net: coin.value(), .. });
    deposit_for_burn::create_deposit_for_burn_with_package_auth_ticket(
        coin, destination_domain, mint_recipient,
        self.relayer,                          // destination caller
        circle_max_fee as u256, min_finality_threshold, vector[],
        CorridorAuth {},                       // CCTP sender = keccak256(type name)
    )
}
```

[TWO WAYS TO CHARGE]

Bridge Kit fee or your own Move fee

Bridge Kit facts from Circle's docs on 6 November. Corridor facts from our contract and runs.

Bridge Kit custom fee

- A computeFee callback in setCustomFeePolicy
- Taken on the source chain by the SDK
- Circle keeps 10 percent of your fee
- No Sui adapter in the docs on 6 Nov

Your Move fee (Corridor)

- Measured from the coin in the burn PTB
- All of it lands in your onchain vault
- CCTP names your package as the sender
- The relayer can prove each fee was paid

[GASLESS FOR THE USER]

Who pays for what

The user holds USDC and nothing else. Measured on our transfers, 8 November.

Cost	Who pays	How
Sui gas for the burn	Corridor operator	Sponsored PTB, gas from its address balance
Corridor fee	User, in USDC	0.3 percent, min 0.01, kept in the vault
Circle fee	Nobody (standard)	Fast into Sui: about 1 bp, out of the mint
Base gas for the mint	Relayer	receiveMessage; it is the destination caller
Sui gas for an inbound mint	Corridor operator	Four-call receive PTB
What the user needs	USDC only	Our test wallet: 0 SUI, 0 ETH

[API/_LIB/RELAY.TS]

A relayer with no database

Abridged. Every call reads Circle and the chain again, so it can be repeated forever and never pays twice.

TYPESCRIPT

```ts
export async function relayOut(digest: string) {
  const m = await irisMessage(SUI.domain, digest);
  if (!m) return { stage: 'attesting' };          // 404 = not indexed yet
  if (m.status !== 'complete') return { stage: 'attesting' };
  if (!eqAddr(body.messageSender, corridorSender()))
    return { stage: 'rejected' };                 // not ours, we do not pay
  if (await usedNonces(m.eventNonce) !== 0n)
    return { stage: 'minted', destinationTx: await findEvmMint(m.eventNonce) };
  const { request } = await base.simulateContract({ functionName: 'receiveMessage',
    args: [m.message, m.attestation], account: relayer.account, .. });
  if (await base.getBalance(relayer) < MIN_RELAYER_WEI)
    return { stage: 'ready' };                    // attested, relayer needs gas
  const hash = await relayer.wallet.writeContract(request);
  return { stage: 'minted', destinationTx: hash };
}
```

[A TRANSFER, START TO FINISH]

Half done is a state, not an error

What the app shows at each stage. Close the tab anywhere: paste the digest and it picks up.

01

Burned

The Sui digest is the receipt. Fee in the vault, USDC burned.

02

Attesting

Iris answers 404 or pending. Show pending, never failed.

03

Ready

Attested. Anyone can mint; our relayer is about to, or needs gas.

04

Minted

usedNonces is set. Repeat calls return the same mint.

05

Recover

Any tab, any day: paste the digest, the relayer reads the chains.

[VERIFIED ON TESTNET]

Proof: our transfers

Sui testnet and Base Sepolia, 8 November 2026. The user wallet held 0 SUI and 0 ETH throughout.

	Chain	Transaction
Sponsored burn, 0.25 in, 0.01 fee	Sui testnet	`DpUY4FYuUbgNHCAxW7oqVeKzdwC9ZNCwkEivVP2imCBF`
Same, through the app (pnpm e2e)	Sui testnet	`2ZRMaMbdC7D5RPZaPEZgHNXkFAca4gVq7LrZjLRLoL9X`
Same, on the live site	Sui testnet	`DBywjeDodSV9FAMvddLzVFCPAQ9BKpGMvjSHvfDFWsGb`
Burn to attestation in 5 seconds	Sui testnet	`C1qH9SEzWwDHPCHf7dVWKiTcNAXuyxGsrRe5HqYMbB5T`
Relayer mints on Base	Base Sepolia	`pending: relayer funding, rerun pnpm relay`
Base to Sui, relayer pays SUI	Sui testnet	`pending: demo Base wallet funding`

[GOTCHAS]

Six things that will bite you

All of them came up while building and running Corridor.

JSDC handler,
. V1 code and
il on V2.

02 No Published.toml

Vendor Circle's sources, write the testnet addresses yourself, strip their inline tests.

03 Sender is a hash

Package auth writes keccak256 of the witness type name, not your package id.

04 Caller cuts both ways

Only your relayer can mint. Lose its key and those transfers can never be minted.

05 Sponsor is a signer

Allow only your calls, one send, no sponsor-owned inputs, a gas cap.

06 Empty relayer, odd error

Base RPC said "Missing or invalid parameters" at 0 ETH. Check the balance, report Ready.

[BUILD IT]

Run Corridor yourself

Node 22+, pnpm, the Sui CLI for the Move part, testnet USDC from faucet.circle.com.

SHELL

```
git clone https://github.com/Nuel-osas/day2-corridor
cd day2-corridor && pnpm install
cp .env.example .env   # your testnet keys
pnpm dev               # http://localhost:5293
pnpm move:test         # 10 Move tests
pnpm send 0.2          # Sui → Base Sepolia
pnpm inbound 0.5       # Base Sepolia → Sui
pnpm relay out <digest>

# your own corridor
cd move/corridor && sui client publish
pnpm open-corridor 100 10000   # 1%, min 0.01
```

01 **Get USDC**
faucet.circle.com, Sui testnet

02 **Quote**
Fee and net, as the Move computes it

03 **Bridge**
Sign with 0 SUI; Corridor pays

04 **Watch**
Burned, attested, minted

05 **Recover**
Reload or paste the digest

[TAKE HOME]

Circle moves the dollars. You run the corridor.

A fee Move enforces.
Gas the user never sees.

CCTP V2 is live on Sui and a Sui burn is attested in seconds. Put your fee in the same PTB as the burn, let CCTP name your package as the sender, pay both gas bills, and keep the relayer stateless so a half-done transfer is always safe to finish.